

- While creating the job or under the configuration tab, there are some options

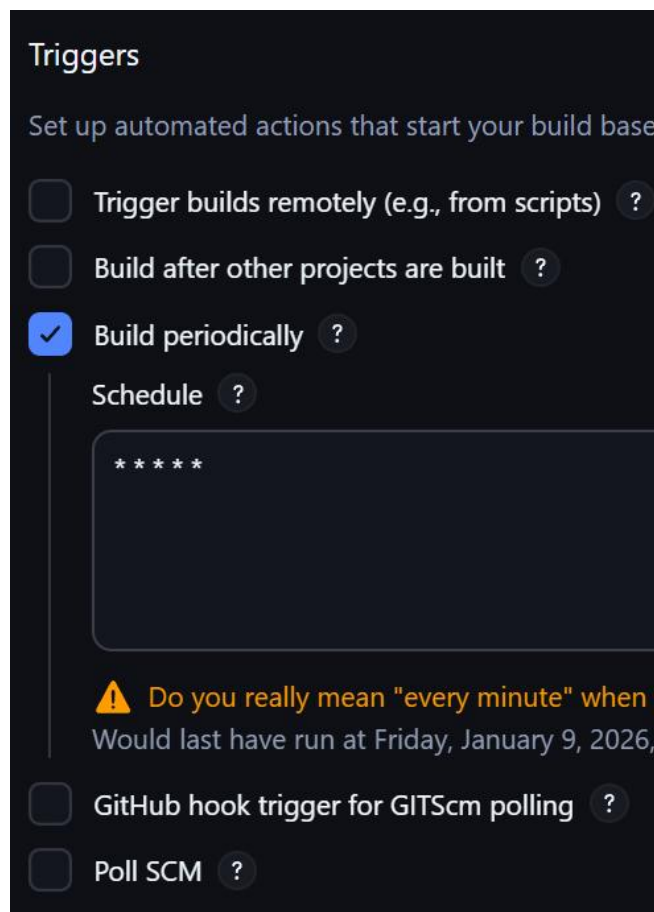


A dark-themed screenshot of a configuration panel with five unchecked checkboxes, each followed by a question mark icon:

- ☐ Discard old builds ?
- ☐ GitHub project
- ☐ This project is parameterized ?
- ☐ Throttle builds ?
- ☐ Execute concurrent builds if necessary ?

^

- ^ This project is parameterized will allow you to set some variables for your job.



A dark-themed screenshot of the 'Triggers' configuration section. It includes a title 'Triggers', a subtitle 'Set up automated actions that start your build base', and several options:

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☒ Build periodically ?

Below the 'Build periodically' option is a 'Schedule' section with a text input field containing '*****'. A warning message is displayed below the input field:

⚠ Do you really mean "every minute" when ?
Would last have run at Friday, January 9, 2026,

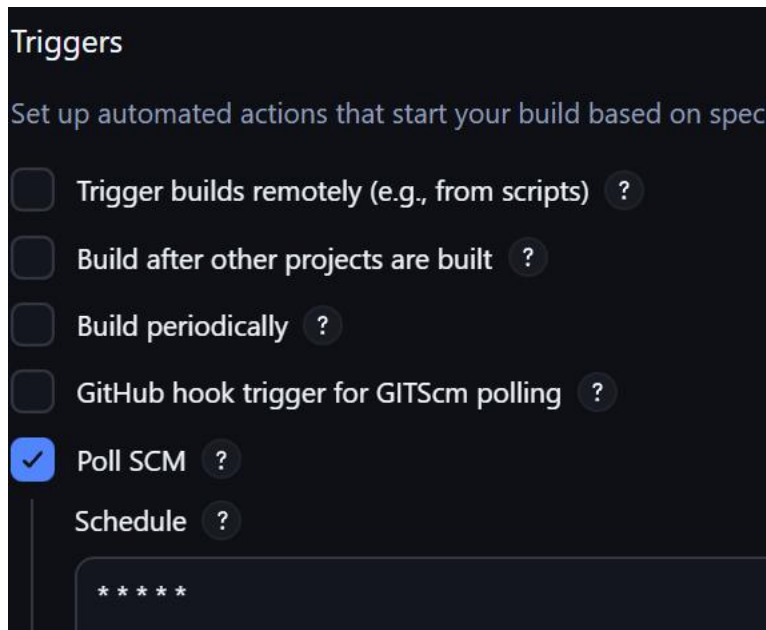
At the bottom of the section are two more unchecked options:

- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

➤

- ^ Build periodically allow you to write cron-job to build this job automatically in some particular intervals.

- One more thing is there **Pole SCM**

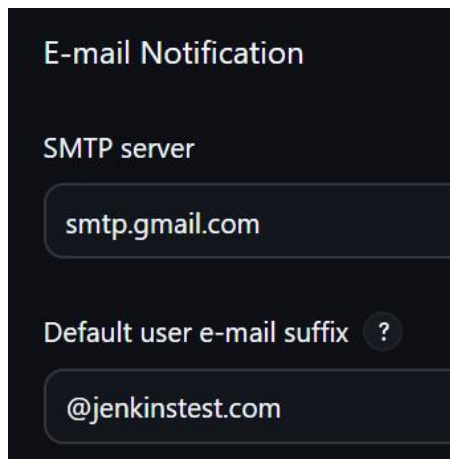


The screenshot shows the 'Triggers' section of the Jenkins configuration interface. It has a dark theme. The title 'Triggers' is at the top. Below it is a subtitle: 'Set up automated actions that start your build based on specific events'. There are five checkboxes with labels and help icons (circles with question marks):

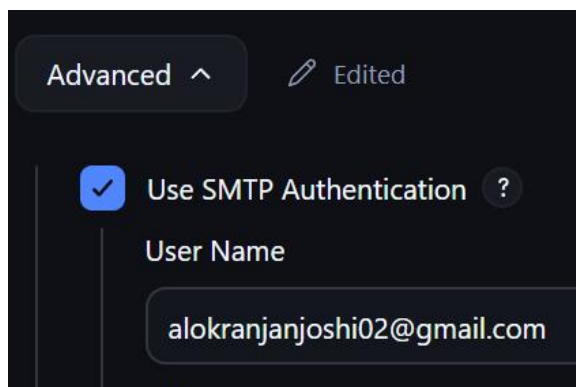
- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☐ GitHub hook trigger for GITScm polling ?
- ☒ Poll SCM ?

Below the checkboxes is a section for 'Schedule' with a help icon. A text input field contains the cron expression: '* * * * *'.

- ⌘ I have mentioned the **cron** for *every minute* it means it'll check the github repo **every minute** and if someone pushes something there then it'll build.
 - ✦ If there is no changes, then it'll not run the build.
- You can also send **email** after the build is completed (pass and fail in both the case)
 - ⌘ We'll use Gmail SMTP

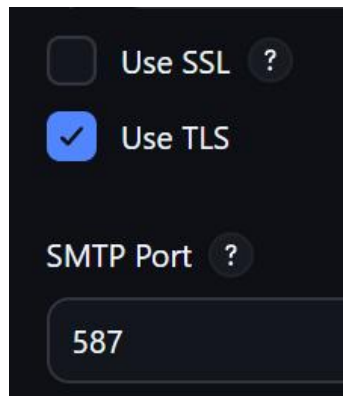


The screenshot shows the 'E-mail Notification' section of the Jenkins configuration interface. It has a dark theme. The title 'E-mail Notification' is at the top. Below it is the 'SMTP server' label and a text input field containing 'smtp.gmail.com'. Below that is the 'Default user e-mail suffix' label with a help icon and a text input field containing '@jenkintest.com'.



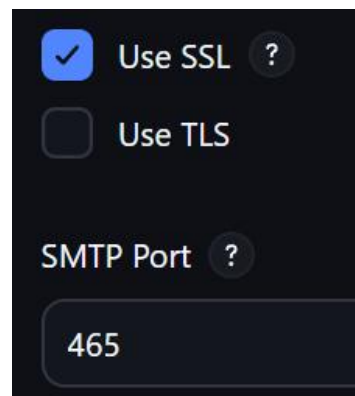
The screenshot shows the 'Advanced' section of the Jenkins configuration interface. It has a dark theme. At the top, there is a tab labeled 'Advanced' with an upward arrow and a link labeled 'Edited' with a pencil icon. Below this is a checkbox labeled 'Use SMTP Authentication' with a help icon, which is checked. Below the checkbox is the 'User Name' label and a text input field containing 'alokranjanjoshi02@gmail.com'.

- You can create app password in by going to your gmail account > **manage gmail account** > **security** then search for **app passwords**



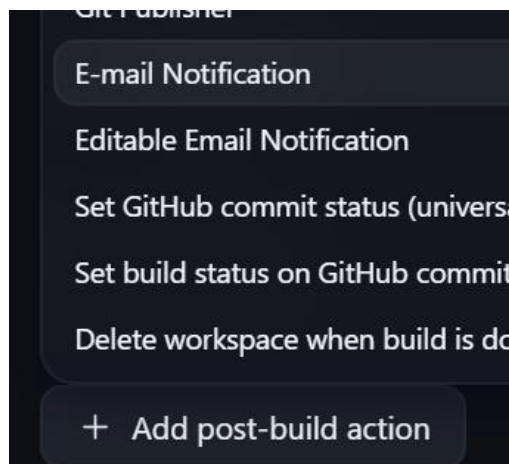
☐ Use SSL ?
☒ Use TLS
 SMTP Port ?
 587

(or)



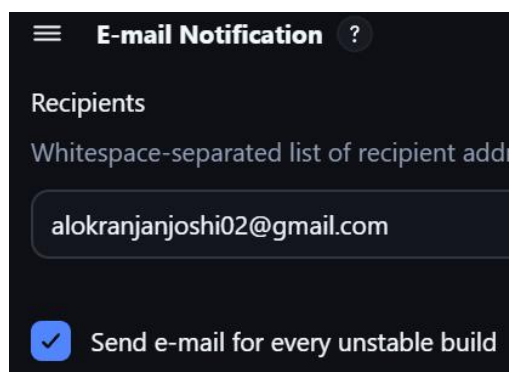
☒ Use SSL ?
☐ Use TLS
 SMTP Port ?
 465

- Then go to your **job** and then **configuration**; under **post build action**, you need to mention these



E-mail Notification
 Editable Email Notification
 Set GitHub commit status (universal)
 Set build status on GitHub commit
 Delete workspace when build is done
 + Add post-build action

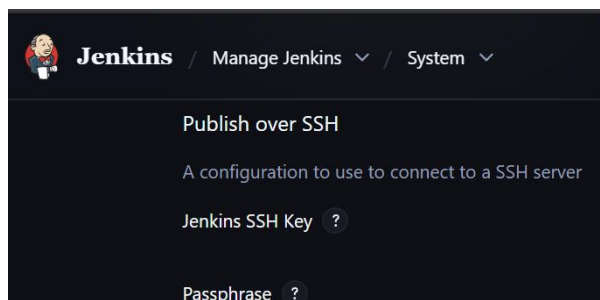
(Email Notification



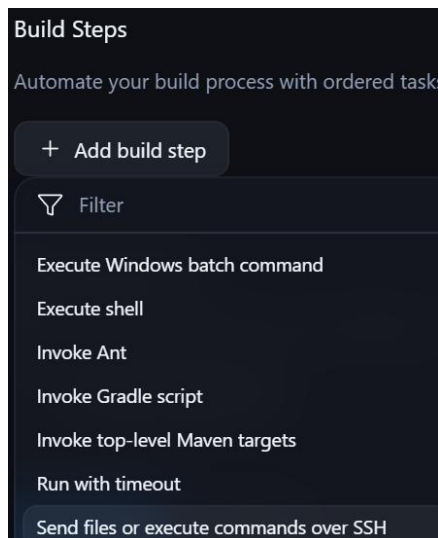
E-mail Notification ?
 Recipients
 Whitespace-separated list of recipient addresses
 alokranjanjoshi02@gmail.com
☒ Send e-mail for every unstable build

- If the build will fail, it'll send a email; and after that if the build passes then it'll send the email.
- It'll not send mail for every passed builds. Just for the failed builds and the **first passed build** (after that failed builds).

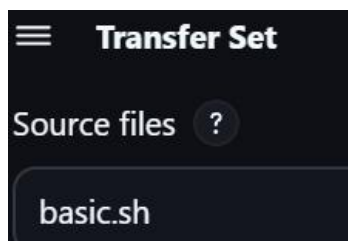
- **NOTE** -----
- The **System** page is a collector page where each installed plugin can inject its own global configuration section.
- Plugins provide build steps; jobs/pipelines use them.
- -----
- I wanted to do **ssh** in jenkins to another server, so I installed the plugin named “**Publish over SSH**” and now, I got one option to add the *global configuration* for that plugin in the **System** page.



- Now you'll get a net option in the **build steps** inside your job's configuration



- This plugin i.e. **Publish over SSH** can only access the files present in the Job's workspace and only supports relative path from that workspace.

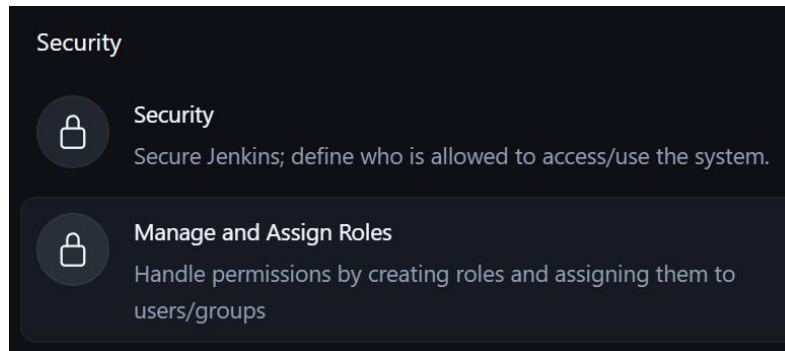


- (like this)
- `basic.sh` is present inside `/var/lib/jenkins/workspace/remote-job`

➤ If you want to give different types of roles to different types of user in Jenkins then install the plugin: **Role-based Authorization Strategy**

↗ After that **manage jenkins > security > authorization** > select Role-Based Strategy

↗ Now you'll get a new option in the **manage jenkins** page

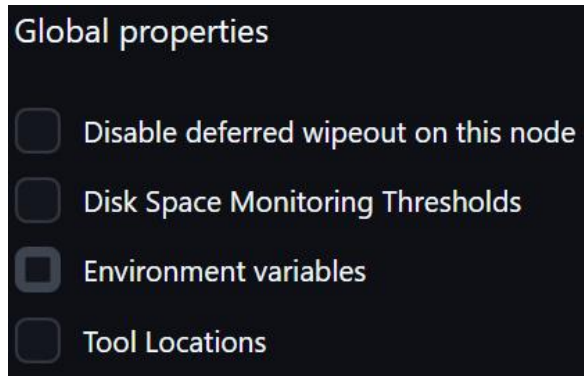


↗

↗ **Manage and Assign Roles**

- You can create global environment variables that can be used by all the jobs

- ⌘ Under **System** page you'll get the below option



- ⌘ Select **Environment variables** and add your environment variables

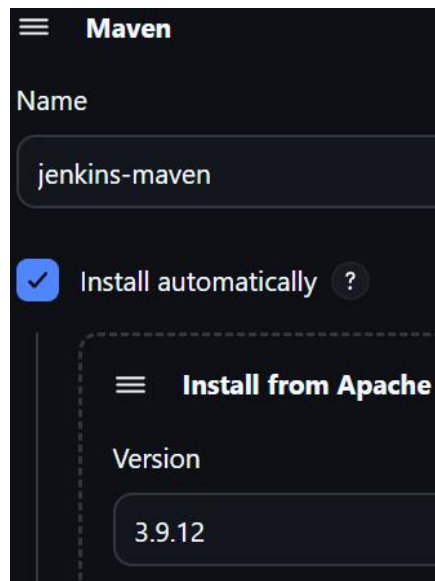
- Jenkins can only auto-install tools that are implemented as Jenkins “Tools” by plugins.
- ↪ Anything that is an OS-level dependency or needs root access is OUTSIDE Jenkins’ control.
- ↪ OS level software that requires the following cannot be auto-installed by Jenkins
 - ☛ root / sudo
 - ☛ Package managers (yum, dnf, apt, apk)
 - ☛ Kernel / System access
 - ☛ Examples:
 - * ❌ Java (system JDK)
 - * ❌ Docker Engine
 - * ❌ Podman
 - * ❌ Git (system git)
 - * ❌ curl / wget
 - * ❌ unzip / tar
 - * ❌ system libraries (glibc, libssl, libstdc++)
 - * ❌ systemd services
 - * ❌ firewall rules
 - * ❌ kernel modules
- ↪ Because, Jenkins run as a non-root user (jenkins user)
- ↪ Here it is being said that it cannot install Java, but Jenkins can install JDK by its own;
- ↪ Whatever Jenkins install, it stores inside the `/var/lib/jenkins/tools/` directory and it is only used to run the pipeline.

➤ **NOTE** -----

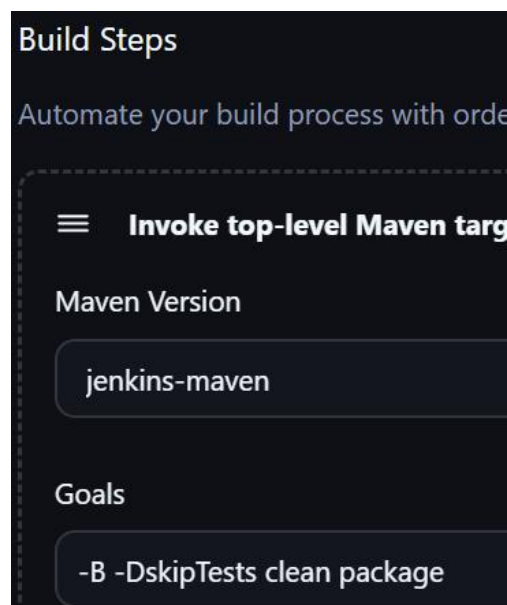
- It doesn't install the tools at the OS level, means when it installs JDK-17 by its own, means it'll be there inside that **tools/** directory and can be used to run the pipeline using that; but you cannot go to the server and run any Java application using that JDK-17.
 - ⌘ It is only used by Jenkins
- JDK should be installed by you at the beginning, because Jenkins is nothing but a JAR file. To run Jenkins you need to install JDK in your system.
 - ⌘ After that to run the pipeline, Jenkins will automatically install JDK by its own.
 - ⌘ Your installed JDK is OS level, Jenkins created JDK is only for pipelines.
- When plugin is installed, Jenkins registers what tools are required for that plugin. And then exposes those tools configuration section in the **tools** page.
 - ⌘ Some tools are provided by Jenkins core, ex: JDK, Maven, ..etc.
 - ⌘ Even if you don't install plugins for those, it'll be there in the **tools** page by default.



- I installed **maven integration** plugin and set the **maven** in the **tools** page
 - ⌘ (note: this maven tool is there by default by Jenkins core; not because of maven integration plugin)



- ⌘ Now this name “**jenkins-maven**” is the alias for maven version 3.9.12
- ⌘ Now inside the **configuration** page of any job, we can use this



- ⌘ Here, don't need to write **mvn** for the commands, it'll be executed with prefix **mvn** by default.

- You can write **groovy** syntax in the Pipeline

```
stage('Build') {  
    steps {  
        echo 'Hello Build'  
    }  
}
```

- ☞ Here **echo** is groovy's syntax.

```
stage('Build') {  
    steps {  
        sh '''  
            echo Build-step-1  
            echo Build-step-2  
        '''  
    }  
}
```

- ☞ Here **echo** is the bash syntax. As we have written inside **sh**

- F

- F

- F

- F

- F

- F

- F

- F

-

-

- ☞ F

- ☞